

GST BILLING & INVENTORY MANAGEMENT
SYSTEM

SCIENCE HUB

The Complete Technical & Business Documentation
("The Science Hub Bible")

Prepared for engineering, operations, and business stakeholders

Fourth Edition | Updated 2026-08-31 | Verified directly against
the live source code

d:\nextApps\science-hub — Next.js 16 · Prisma · PostgreSQL (Neon) · Vercel

Table of Contents

1	Executive Summary
2	High-Level System Overview
3	Complete Technology Stack
4	Folder Structure
5	Application Flow
6	User Roles & Permissions
7	Authentication & Sessions
8	Database (Full ER Reference)
9	Complete Module Documentation
10	Invoice Flow — Step by Step
11	Stock Flow & the Movement Ledger
12	GST Logic
13	API Documentation (Full Reference)
14	Frontend Architecture
15	Backend Architecture
16	Data Flow
17	PDF Generation
18	Email System
19	Security
20	Error Handling
21	Deployment
22	Third-Party Services
23	Environment Variables

24 Project Dependencies

25 Business Workflow (A Day in the Life)

26 Non-Technical Explanation

27 Developer Guide

28 Future Improvements

How this document was built

Every claim in this document was verified by reading the actual source code at `d:\nextApps\science-hub` — not guessed, and not copied blindly from the project's own internal notes (`CLAUDE.md` , `DEVELOPMENT_LOG.md`). Places where those internal notes were found to be outdated, and later corrected, are called out explicitly in "Update" callouts throughout, so this Bible reflects the application as it truly runs today, **2026-08-31**. This is the same methodology the Third Edition established, carried forward unchanged: internal notes (`CLAUDE.md` , `DEVELOPMENT_LOG.md` 's Era 12/13) were used as a map of where to look, then every load-bearing claim about cache invalidation, idempotency, dependency versions, and the in-progress formula-injection work was spot-verified directly against the relevant source file before being written down here.

What's new in this Fourth Edition (since the Third Edition, 2026-08-17)

Ten days of work landed across seven commits (2026-08-18 → 2026-08-27), the largest concentration of fixes this Bible has documented in a single edition — plus one further batch of changes sitting uncommitted in the working tree as of 2026-08-31. In order:

- **Input validation gained real length bounds** (commits `9de3a12` / `f48857a` , 2026-08-18) — every `rules.*` validator and per-entity `validateXInput()` in `src/lib/validation.ts` now enforces server-side min/max length checks (name, address, city, bank fields, password) alongside the format rules that already existed, closing a real gap where an empty, absurdly long, or whitespace-only value could reach the database even though the client-side form already blocked it visually. Alongside it: the topbar condenses into a `MoreMenu` below 480px, a mobile-card skeleton was added to the Permissions page, and stale form drafts now expire instead of silently resurrecting a form the user had already dismissed empty.
- **A full audit-fix pass plus a PDF renderer rewrite** (`3cea3f8` / `f1878aa` , 2026-08-19) — closed nine categories of findings from `docs/SCIENCE_HUB_AUDIT_REPORT.md` (`AUTH-001/002` , `API-001..004` , `DB-001` , `BIZ-001..003` , `SEC-003..005` , `PDF-002` , `UX-001`) — see Chapter 19 for the security-relevant subset — and reworked the invoice/purchase-bill PDF renderer. A same-day follow-up fixed a genuine Vercel build failure caused by `next-auth` / `@auth/core` peer-dependency on `nodemailer ^7|^8` , which conflicted with the CVE-driven bump to `nodemailer@9.0.5` — resolved via an `npm overrides` block rather than downgrading either package (see Chapter 24).
- **Drag-to-reorder line items and product bulk import** (`0e483eb` , 2026-08-20) — invoice, purchase-bill, and rate-list item tables all gained drag-to-reorder; Products gained a bulk-import path from Excel/CSV, mirroring the existing Rate List import parser's approach. `ConfirmDialog` now portals to `document.body` (it had been rendering trapped inside a

transformed ancestor); the admin activity log now uses the shared `Pagination` component instead of a hand-rolled one; the product form's "Price" field was relabeled "List Price" for clarity against `purchasePrice`.

- **The full 14-phase, app-wide audit** (`66177dc` , 2026-08-21) — 25+ findings closed across authentication/API trust boundaries, blob-URL validation, the same-day payment-date bug, N+1/unbounded queries, missing error boundaries, and accessibility gaps; dependency bumps cleared every critical/high `npm audit` finding; a retrying `prisma migrate deploy` wrapper was added for Neon cold-start flakiness. This is already documented in full as `CLAUDE.md` Feature #36 and is summarized by category in Chapter 19 and Chapter 24 below.
- **Shared toolbar components and a second mobile-breakpoint pass** (`7fbad8e` / `ed66e57` / `d7fc2bb` , 2026-08-25) — four new shared components (`SearchField` , `DateRangeFilter` , `HeaderActionsRow` , `ToolbarField`) replaced hand-duplicated toolbar markup across every list/report page; `Spinner` overlays now portal to `document.body` the same way `ConfirmDialog` already does; mobile form-control heights were synced across `Button` / `Input` / `Select` / `DatePicker` / `PasswordInput` / `PhoneInput` ; the dashboard's bar-chart value labels were reworked to float above each bar rather than sit in a shared row that clipped against the chart's own scroll container; and several smaller layout fixes landed (Purchase Bills list column widths, draft/numbering `InfoBanner` placement, overview chart top padding).
- **Cache-invalidation, idempotency, and data-integrity hardening** (`76412b3` , 2026-08-26) — the single most substantively important commit in this window. A real user report (a restored credit note not appearing on the Credit Notes list without a hard refresh) traced to a systemic bug in `bustCachePrefix()` itself, not a one-off missing cache-bust call — see Chapter 16.1 for the full mechanism. The same commit also added Purchase Bill edit's missing optimistic-concurrency recheck, client-generated idempotency keys across every money- and document-creating route, moved two report aggregations onto the database, capped the Bin's unbounded GST-retained-document fetch, and added three new date-column indexes — see Chapter 9 and Chapter 13 for the specifics.
- **Unguarded fetch responses fixed on 404** (`baef4f2` , 2026-08-27) — Purchase Bill detail/edit and Vendor edit were treating any fetch response body as valid entity data without checking `r.ok` first, so a deleted/invalid id crashed to the error boundary instead of showing a proper not-found state. Found through live browser testing, not a code-review pass — see Chapter 20.

IN PROGRESS — NOT YET COMMITTED AS OF 2026-08-31

Two further changes sit in the working tree, uncommitted: **CSV/XLSX export formula-injection hardening** (a new `src/lib/formulaSafety.ts` , `neutralizeFormulaCell()` , wired into `gstFilingWorkbook.ts` , `gstFilingZip.ts` 's CSV writer, and `xlsxExport.ts` — see Chapter 19.8) and a **stock-adjustment preview UI** on the Product detail page's Adjust Stock dialog (current/delta/new-stock figures with a color tone per direction — see Chapter 9.3). Both are described below exactly as they exist in the working tree right now, flagged wherever they're mentioned, and should not be treated as shipped, stable, or reverted-proof until a future edition confirms they landed on a committed branch.

Nothing in the Third Edition's core architecture changed: still every page `"use client"` , still no `"use cache"` / `cacheComponents` , still the same 21-model schema, still the same GST math, still the same document-numbering system. This edition is a hardening and polish pass on top of a feature set that was already largely complete as of the Third Edition — see Chapter 9 for confirmation that Rate Lists, configurable document numbering, transport charges, and GST-safe bin retention remain accurately described and unchanged in shape.

1 Executive Summary

What is Science Hub?

Science Hub is a web-based business management system built for a company that sells laboratory and scientific supplies (chemicals, glassware, instruments, safety equipment, consumables). In plain terms, it is the digital replacement for the paper ledgers, Excel sheets, and manual invoice books a supplies business would otherwise use to run itself day to day.

It covers the full commercial cycle of such a business:

- Keeping a catalogue of products, their brands, categories, prices, and how much stock is on the shelf.
- Creating GST-compliant sales invoices for customers, automatically calculating the correct tax.
- Recording payments as customers pay, and knowing at a glance who still owes money.
- Handling product returns from customers by issuing credit notes.
- Buying stock from vendors/suppliers, recording purchase bills, and paying those bills — now emailable directly to a vendor or their accountant, just like an invoice.
- Keeping an audit-proof ledger of every single stock movement — every sale, purchase, return, edit, and cancellation.
- Producing management reports — sales, purchases, outstanding balances, GST summaries, and a ready-to-file GST return package.
- Building and sharing standalone price-sheets/rate lists with customers, independent of the product catalogue.
- Emailing professional PDF invoices, purchase bills, and rate lists directly to customers or vendors.
- Controlling who on staff can see or touch what, with a full audit trail of every action anyone takes.
- A "recycle bin" so that accidental deletions are never permanently lost — indefinitely for legally-numbered documents (invoices, purchase bills, credit notes), 30 days for everything else, now shown with a capped, "load more"-driven view rather than an unbounded fetch of the whole retained history.

What business problem does it solve?

A scientific-supplies trading business deals with GST tax rules, stock that must never be oversold, customers who pay in instalments, and a legal requirement to keep clean paperwork for tax filing. Doing this by hand (or in loose spreadsheets) is slow and error-prone: invoice numbers get duplicated, stock counts drift from reality, GST math is done inconsistently, and there is no record of who changed what. Science Hub solves all of this by making every one of these steps a single, auditable, automatically-calculated action inside one system — and, as of this edition, one that is now considerably more resistant to a duplicated network retry silently creating a second invoice, a stale browser cache showing an out-of-date list, or a race between two people editing the same purchase bill at once.

Who uses it?

Roles at a glance

Role	Who they typically are	What they can do
ADMIN	Business owner / senior manager	Everything — full read/write access to all data, user management, business settings, bank details, document-numbering configuration, permanently deleting records, GST filing, and manual stock adjustments.
STAFF	Day-to-day billing/accounts clerk	Full read/write on invoices, customers, products, purchases, payments, rate lists, etc. Can be individually granted or denied access to six specific "sections" (overview dashboards, reports, and payment histories) by an admin.
MANAGER	Supervisor who needs visibility but not editing rights	Same section-based visibility model as staff, but is blocked from creating, editing, or deleting anything anywhere in the system — a strict read-only viewer.

Why was it built?

The codebase and its own internal documentation make clear this is a purpose-built, single-tenant system for one specific business (not a generic multi-company SaaS product) — the system is deployed once, for one organisation, on Vercel with one Neon database.

Main features (at a glance)

- Invoice creation, editing, and soft-deletion, numbered in an admin-configurable layout (Settings → Document Numbering) rather than one fixed shape.
- Automatic GST computation (CGST+SGST for in-state sales, IGST for inter-state sales), server-verified independently of the browser.

- An optional per-document transportation/freight charge line, with its own GST rate and amount, shown separately from the item-level tax on both invoices and purchase bills.
- Product returns issued as numbered credit notes, same configurable-layout numbering as invoices.
- Purchase bills from vendors with the same configurable numbering, attachments, a cancel/un-cancel workflow, and (new this edition) their own dedicated email-send route.
- A single, append-only stock movement ledger recording every reason stock ever changed.
- Rate Lists — a standalone, downloadable price-sheet/catalogue builder for sharing quotations with customers, deliberately independent of the Product catalogue, with bulk import, PDF generation, email send, and Excel export.
- Drag-to-reorder line items on invoices, purchase bills, and rate lists (new this edition).
- Bulk import of products from Excel/CSV (new this edition), on the same pattern Rate Lists pioneered.
- Section-based permissions so an admin can fine-tune what a given staff/manager account can view.
- Recycle Bin — 30-day grace period and automatic purge for most record types; invoices, purchase bills, and credit notes are retained indefinitely, now paginated with a "Load more" control instead of always fetching the entire retained history in one request.
- Global search across invoices, customers, products, vendors, purchase bills, brands, and categories.
- A dedicated GST filing package (Sales Register, Purchase Register, Credit Notes, HSN Summary) exportable as a ZIP or `.xlsx`, with CSV/XLSX formula-injection hardening currently sitting in the working tree, not yet committed (see Chapter 19.8).
- Business branding (name, tagline, logo) that appears on invoices, rate lists, the login page, and the browser tab.
- Full activity log of every meaningful action, by whom and when.
- Email delivery of invoice, purchase-bill, and rate-list PDFs straight from the app via Gmail.
- Client-generated idempotency keys on every document- and payment-creating route (new this edition), so a duplicated network retry can no longer create a second row.
- A default-deny API gatekeeper (`middleware.ts`) protecting every backend route as a safety net alongside each route's own permission check.

Benefits

- **Accuracy** — GST, discounts, and stock arithmetic are computed once, in one place, the same way every time (never re-typed by hand).

- **Accountability** — every create/edit/delete is attributed to a named user and time-stamped in the activity log.
- **Safety net** — soft-delete and the recycle bin mean mistakes are recoverable, not catastrophic, with extra protection for the documents that legally must not silently vanish.
- **Compliance-readiness** — the GST filing package is built directly from the same transactional data used to run the business, so there is one source of truth rather than a reconciliation exercise at tax time.
- **Access control that matches how a small business is actually staffed** — an owner can hand out narrowly-scoped, read-only, or fully-privileged access without needing separate software.
- **Resilience against duplicate submissions and stale UI** (new this edition) — idempotency keys and a fixed cache-invalidation bug together close a class of bugs where a slow network or a page that didn't refresh could produce a wrong number, a duplicate row, or a list that silently disagreed with reality.

2 High-Level System Overview

At the simplest level, Science Hub works like almost every modern web application: a person uses a web browser, that browser talks to a server over the internet, the server checks a database, and sends the answer back to be displayed. Below is that same idea, specific to how this application is actually built.

- 1 User opens the Science Hub website in their browser (e.g. on Vercel).
- 2 The browser downloads the page's React code (built with Next.js).
- 3 The page checks: is this user logged in? (NextAuth session cookie)
- 4 The page calls an internal API (e.g. `/api/invoices`) to fetch or save data.
- 5 The API route checks permissions, validates the request, and talks to the database via Prisma.
- 6 The database (PostgreSQL, hosted on Neon) reads or writes the requested rows.
- 7 The API sends a JSON response back to the browser.
- 8 The page updates on screen without a full reload, and its local cache is refreshed — now via a cache-busting mechanism that correctly reaches every related sub-route, not just the exact URL a page happens to own (see Chapter 16.1).

Everything the user sees is rendered "client-side" — every page in this application is explicitly marked `"use client"`, meaning the browser itself does the work of building the visible page after fetching data, rather than the server pre-building the whole page. This remains a deliberate architectural choice, made to avoid a specific bug pattern this version of Next.js has with server-rendered/cached data (`"use cache"` and `cacheComponents: true` are both confirmed, as of this edition, to still be forbidden in this codebase — see Chapter 3).

The three big layers

Layer	Technology	Responsibility
Presentation (Frontend)	Next.js 16 (App Router) + React 19	Everything the user sees and clicks — forms, tables, dashboards, PDFs on screen.
Application (Backend)	Next.js API Routes (same project)	Business rules: validation, GST math, document numbering, the stock ledger, permission checks, email sending, idempotency checks.
Data (Database)	PostgreSQL on Neon, accessed via Prisma ORM	The permanent record of every customer, product, invoice, bill, rate list, and user.

Note: because both the frontend and backend live in the same Next.js project, there is no separate "backend server" to deploy — Vercel builds and hosts both together as one unit.

3 Complete Technology Stack

Every technology below is confirmed present in `package.json` or the source code directly — nothing in this list is assumed.

Core Framework

Technology	What it is	Why it's used here
Next.js 16	A framework built on top of React that handles routing, bundling, and server/API code, all in one project.	Lets one project serve both the web pages and the API that powers them, deployed as a single unit on Vercel.
React 19	A JavaScript library for building interactive user interfaces out of reusable "components".	Every screen in Science Hub — invoice forms, rate-list builders, dashboards, tables — is built from React components.
TypeScript	A version of JavaScript that adds type-checking.	Used throughout the codebase for safer, self-documenting code.

Data & Storage

Technology	What it is	Where/why it's used
PostgreSQL	A mature, industry-standard relational database.	The single source of truth for every business record in Science Hub.
Neon	A cloud hosting service that runs PostgreSQL for you.	Hosts the production database; connected via a pooled connection string. Vercel builds now run a retrying <code>migrate deploy</code> wrapper (<code>scripts/migrate-deploy-retry.js</code> , since 2026-08-21) to absorb Neon's cold-start latency.
Prisma	An "ORM" that lets developers write database queries as regular code.	<code>prisma/schema.prisma</code> defines all 21 data models; almost every API route uses the generated Prisma client directly.
Vercel Blob	A cloud file-storage service for storing uploaded files.	Stores purchase-bill attachments and the uploaded business logo.

The schema remains at 21 models, unchanged in count since the Third Edition — no new model was added in this window. `package.json` 's build script remains `node scripts/migrate-`

`deploy-retry.js` && `next build` (the retry wrapper around `prisma migrate deploy`, added 2026-08-21 — see Chapter 21). Three fields (`Invoice.balanceDue` , `PurchaseBill.balanceDue` , `Product.isLowStock`) remain real Postgres `GENERATED ALWAYS AS (...) STORED` columns.

Authentication & Security

Technology	What it is	Where/why it's used
NextAuth v4	A ready-made authentication library handling login sessions.	Configured in <code>src/lib/auth.ts</code> with an email+password ("Credentials") login and JWT sessions lasting 8 hours.
bcryptjs	A one-way password-hashing algorithm.	Used to hash every stored user password (bcrypt cost factor 12).
Node's built-in <code>crypto</code> module (AES-256-GCM)	A strong, industry-standard symmetric encryption algorithm.	Encrypts secrets at rest: the Gmail app password and the bank account number, in <code>src/lib/crypto.ts</code> .

Communication & Documents

Technology	What it is	Where/why it's used
Nodemailer	A Node.js library for sending email.	Sends invoice, purchase-bill (new this edition), and rate-list PDFs to recipients via Gmail SMTP. Bumped to <code>9.0.5</code> in this window — see Chapter 24.
jsPDF	A JavaScript library that builds a PDF file entirely inside the browser.	Assembles the final invoice, credit-note, purchase-bill, and rate-list PDF files via one shared client-side generator (rewritten this window — see Chapter 17).
html2canvas	A library that "photographs" a section of a web page into an image.	Captures the on-screen document layout so it can be pasted, page by page, into the jsPDF document.
ExcelJS	A library for generating and parsing real <code>.xlsx</code> files.	Powers Excel export/import across Credit Notes, Sales/Purchase Reports, Rate Lists, GST Filing, and product bulk-import.
JSZip	A library for building <code>.zip</code> archive files in code.	Bundles the GST filing package into one downloadable ZIP.

Styling & UX

Technology	What it is	Where/why it's used
CSS Modules	Styles automatically scoped to one component.	Every page/component has its own <code>*.module.css</code> file.
CSS Custom Properties	Reusable named values in CSS that can be swapped instantly.	Defined in <code>src/app/globals.css</code> ; switching light/dark mode or accent colour is just swapping variable values.

Hosting & Deployment

Technology	What it is	Where/why it's used
Vercel	A cloud hosting platform purpose-built for Next.js applications.	Hosts the entire live application; auto-builds/deploys on push.
Environment Variables	Configuration values kept outside the code itself.	Database connection string, auth secret, encryption key, Gmail credentials, app's public URL. Full list in Chapter 23.
npm	Node.js's package manager.	Drives every install/build/dev script. <code>postinstall</code> runs <code>prisma generate</code> and installs the repo's git hooks — never remove it.

Not present in this codebase, still: no Redis (rate limiting remains an in-memory fixed-window counter), no Railway hosting, no AI/Gemini bill-extraction feature.

4 Folder Structure

All application code lives under `src/`. The overall shape is unchanged from the Third Edition — pages live under `app/(dashboard)/.../page.tsx`, the API they talk to lives at the matching path under `app/api/.../route.ts`, and any logic shared by more than one place is pulled into `lib/`. There is deliberately no separate "controllers/services/repositories" layering.

```

src/
  app/
    (dashboard)/          A route "group" -- every page a logged-in user sees, sharing one
    dashboard/            The home KPI dashboard
    sales/                Customers, Invoices, Payments Received, Credit Notes, Rate Lists
    purchases/            Vendors, Purchase Bills, Payments Made, Purchases Overview
    products/, brands/, categories/  The product catalogue
    reports/              Sales Reports, Purchase Reports, GST Reports
    admin/                User management, activity log, section-permissions manager
    bin/                  The Recycle Bin
    settings/             Business identity, bank details, Gmail, logo, terms & conditions
  api/                    Every backend endpoint -- one route.ts file per URL path (Ch. 13)
                           send-purchase-bill/ route (2026-08-26)
  login/, forgot-password/, reset-password/, find-email/        Public, unauthenticated pages
  layout.tsx              The root page shell
  providers.tsx            Wraps every page in Session / Branding / Theme / Toast context
  components/
    ui/                    Reusable building blocks: Button, Input, Select, UnitCombo, Badge,
                           PdfPreviewModal, and (new this edition) SearchField, DateRangeField,
                           ToolbarField
    layout/                DashboardShell, GlobalSearch, Breadcrumb
    dialogs/               Modal, ConfirmDialog (now portals to document.body), PdfCopyDialog
    invoices/, rateLists/, purchases/  Shared per-entity form building blocks
  lib/
    auth.ts, apiAuth.ts, prisma.ts, validation.ts (length-bounded this edition), stockMovement.ts,
    invoiceCalc.ts, purchaseBillForm.ts, roundOff.ts, documentNumbering.ts,
    rateListForm.ts, rateListQuery.ts, rateListImport.ts,
    crypto.ts, activity.ts, rateLimit.ts,
    generateInvoicePdf.ts (rewritten this edition), pdfCache.ts,
    sections.ts, businessBranding.tsx,
    gstFiling.ts, gstFilingZip.ts, gstFilingWorkbook.ts (formula-safety wiring in progress,
    xlsxExport.ts (formula-safety wiring in progress, uncommitted),
    useCache.ts (bustCachePrefix fixed this edition), useIdempotencyKey.ts (new this edition),
    formulaSafety.ts (new, uncommitted)
  types/
    next-auth.d.ts
  prisma/
    schema.prisma          21 models (unchanged count)
    seed.ts
  scripts/
    migrate-deploy-retry.js New (2026-08-21) -- retries "prisma migrate deploy" up to 5x, 5s
    install-git-hooks.js

```

What's new here since the Third Edition

- `src/app/api/send-purchase-bill/route.ts` — a new email-send endpoint mirroring `/api/send-invoice / /api/send-rate-list`, added 2026-08-26 alongside the cache-

invalidation hardening commit. Unlike the other two senders it accepts a comma-separated recipient list (up to 10 addresses), for sending a bill to both a vendor and their accountant in one send.

- `src/lib/useIdempotencyKey.ts` — a small client-side hook minting a stable, per-form-mount UUID sent with mutating POSTs (Chapter 9, 13).
- `src/lib/formulaSafety.ts` — new and **currently untracked/uncommitted** — a single `neutralizeFormulaCell()` helper used by the GST filing workbook/ZIP/CSV writers and the generic Excel exporter.
- `src/components/ui/SearchField.tsx`, `DateRangeFilter.tsx`, `HeaderActionsRow.tsx`, `ToolbarField.tsx` — four shared toolbar components (2026-08-25) that replaced near-duplicate hand-rolled search/date-filter/action-row markup that had been copy-pasted across every list and report page.
- `scripts/migrate-deploy-retry.js` — new (2026-08-21), wraps `prisma migrate deploy` with up to 5 retries, 5 seconds apart, to absorb a Neon free-tier cold-start that previously surfaced as an intermittent, non-deterministic build failure.

5 Application Flow

This is the life of a single, typical user session, beginning to end — unchanged in shape since the Third Edition.

- 1 Visit the site — redirected to `/login` if no valid session.
- 2 Enter email + password — NextAuth verifies against the hashed password in the database.
- 3 A signed session cookie (JWT) is set in the browser, valid for 8 hours.
- 4 Redirected to the Dashboard — KPI cards load from `/api/reports?type=combined-dashboard`.
- 5 User navigates via the sidebar (only sees items their role/section grants allow).
- 6 e.g. Create Invoice → fill form → client validates → submit to `/api/invoices` (POST), now carrying a client-generated idempotency key.

Server re-validates everything, computes GST, decrements stock, writes the invoice — all inside one database transaction, and rejects a repeated idempotency key on the same customer/parent as a duplicate rather than creating a second row.
- 8 Invoice detail page can generate a PDF (client-side), download it, print it, or email it to the customer.
- 9 Every one of the above write actions is recorded in the Activity Log, visible to admins.
- 10 Deleting anything moves it to the Recycle Bin (soft-delete) rather than erasing it.

Flow-level conventions confirmed unchanged from the Third Edition

- Every list screen is paginated server-side, with a matching `/stats` endpoint.
- Every popup dialog in the app is the same `Modal` component, in `variant="fullscreen"`.

What changed this edition

- **List/report toolbars are now built from shared components**, not per-page hand-rolled markup — `SearchField`, `DateRangeFilter`, `HeaderActionsRow`, `ToolbarField` (Chapter 4, 14).
- **A mutating submit now carries an idempotency key** for Invoice, PurchaseBill, Payment, PurchasePayment, and Return creation — a double-tapped Save button or a retried network request is recognized server-side as the same submission (Chapter 9, 13, 16).

- **A page fetching one entity by id must check `r.ok`** before treating the response body as that entity — fixed on Purchase Bill detail/edit and Vendor edit (2026-08-27); any earlier assumption that a JSON body implies success is now known to be wrong and has been corrected at these three call sites (Chapter 20).

A Rate List still follows a lighter version of the same invoice-style flow, and Purchase Bills gained one new step this edition: like an invoice, a purchase bill can now be emailed directly from its detail page via `POST /api/send-purchase-bill`, to one or more comma-separated recipients.

6 User Roles & Permissions

Science Hub has three roles, stored as a plain text field (`User.role`): `admin` , `staff` (the default for new users), and `manager` . This three-role model, and the section-permission system below, are unchanged in shape since the Third Edition and remain verified accurate against both `src/lib/apiAuth.ts` and `src/lib/sections.ts` .

6.1 Admin

ADMIN has unrestricted access to everything. Only an admin can reach: Settings, Admin (user management, activity log, permissions manager), permanently delete anything from the Recycle Bin, run the IFSC bank-code lookup, and bulk-clear ledger history. An admin automatically passes every section-permission check.

6.2 Staff (default role)

STAFF can fully create, edit, and delete the core business records without restriction. What is restricted for staff is visibility into six specific "sections" of the app — see 6.4 below.

6.3 Manager — read-only

MANAGER is functionally identical to staff for section-based visibility, but is hard-blocked from every mutating action in the system. This is enforced in exactly one shared rule (`requireWriteAccess()` on the server, mirrored by `useCanWrite()` on the client).

6.4 Section-based permissions

Section key	What it controls
<code>sales_overview</code>	The Sales Overview dashboard and its KPI widgets on the main Dashboard page
<code>purchase_overview</code>	The Purchases Overview dashboard and its KPI widgets on the main Dashboard page
<code>reports_sales</code>	The Sales Reports page
<code>reports_purchases</code>	The Purchase Reports page
<code>payments_received</code>	The Payments Received history page
<code>payments_made</code>	The Payments Made history page

The GST Reports page and its underlying API require both `reports_sales` and `reports_purchases` together (or admin). This grid is managed from Admin → Permissions; unchanged this edition, apart from a mobile-card skeleton being added to the page (2026-08-18).

6.5 Permission enforcement summary

Check	Enforced where	Effect
<code>requireSession()</code>	Nearly every API route	401 if not logged in at all
<code>requireAdmin()</code>	User management, permissions, settings writes, bin permanent-delete, ledger bulk-clear	403 unless role is admin
<code>requireWriteAccess()</code>	Every create/edit/delete endpoint for core business data	403 if role is manager
<code>requireSectionAccess(section)</code>	Overview dashboards, reports, payment-history APIs	403 unless admin or the section is individually granted

UPDATE — RE-VERIFIED THIS EDITION

`middleware.ts` 's public allowlist is unchanged: `PUBLIC_API_PREFIXES` is `["/api/auth/", "/api/setup"]` and `PUBLIC_API_EXACT` is `={"/api/settings/branding"}`. The new `/api/send-purchase-bill` route is *not* on this list and correctly requires a session plus `requireWriteAccess()` inside its own handler, exactly like `/api/send-invoice` and `/api/send-rate-list` — the audit and hardening work in this window did not widen the public surface anywhere.

7 Authentication & Sessions

7.1 How login works

- 1 User submits email + password on `/login` .
- 2 Email is lowercased/trimmed; NextAuth's "Credentials" provider runs `authorize()` .
- 3 Rate limits checked: max 30 attempts/15min per IP address, and max 8 attempts/15min per account.
- 4 Password checked with `bcrypt.compare()` against the stored hash.
- 5 On success: a signed JWT is issued containing user id, role, and granted sections.
- 6 JWT is stored in an `httpOnly` , secure, 8-hour session cookie.

Whether the email doesn't exist or the password is wrong, the server always still runs a `bcrypt` comparison (against a fixed dummy hash if no such user exists) before responding, so both failure cases take roughly the same amount of time. The 2026-08-12 fix wrapping `authorize()` 's `prisma.user.findUnique()` lookup in its own try/catch remains in place and unregressed as of this edition — confirmed directly against `src/lib/auth.ts` .

7.2 What's inside the session

Unchanged: `id` , `name` , `email` , `role` , `sections` . Section grants reload every 30 seconds; a full account-validity check happens at most every 5 minutes.

7.3 Cookies and JWT

Unchanged — a signed, tamper-proof `httpOnly` , Secure-in-production cookie.

7.4 Password reset flow

Unchanged five-step flow (Forgot Password → always "check your email" → 1-hour token → new password ≥8 chars → old sessions invalidated). A separate "Find my email" page still exists — see Chapter 19.1's note on `AUTH-001` , one of this window's audit findings, which affected this exact page.

7.5 Password storage

bcrypt, cost factor 12. Unchanged.

7.6 Authorization vs. authentication

Unchanged — the four guard functions in Chapter 6.5 remain the single enforcement point.

7.7 The middleware gatekeeper

Unchanged from the Third Edition; re-confirmed this edition (Chapter 6.5's callout above) that the allowlist has not silently grown.

8 Database (Full ER Reference)

The database has **21 tables** ("models" in Prisma) defined in `prisma/schema.prisma` — unchanged in count since the Third Edition. There are no database-level enum types in this schema.

8.1 Soft-delete vs. permanent records

Has a "Recycle Bin" (<code>deletedAt</code> field)	Never soft-deleted (permanent/append-only)
Customer, Category, Brand, Product, Invoice, Return (credit note), Vendor, PurchaseBill, RateList	User, PasswordResetToken, ActivityLog, InvoiceItem, Payment, ReturnItem, BusinessSettings, PurchaseBillItem, PurchasePayment, StockMovement, SectionPermission, RateListItem

8.2 Schema changes this edition

UPDATE — NEW INDEXES, 2026-08-26

Commit `76412b3` added indexes on `Payment.date` , `PurchasePayment.date` , and `Return.date` , closing the last three soft-deletable/date-filterable models that had been missing an index on their own date column (the Third Edition's Chapter 8 already listed the equivalent indexes on `Invoice.date` / `PurchaseBill.billDate` ; these three payment/return-side tables had not caught up until now). This is a pure performance change — no field was added, removed, or renamed.

Separately, the earlier full-app audit (2026-08-21, `DB-001`) added a `deletedAt` index to `Vendor` and `PurchaseBill` , the two soft-deletable models that had been missing one — every other soft-deletable model already had this index as of the Third Edition.

No model gained, lost, or renamed a field in this window; the 21-model count and every field described in the Third Edition's field-by-field walkthrough (User, PasswordResetToken, ActivityLog, Customer, Category/Brand, Product, Invoice, InvoiceItem, Payment, Return/ReturnItem, BusinessSettings, Vendor, PurchaseBill, PurchaseBillItem/PurchasePayment, StockMovement, RateList/RateListItem, SectionPermission) remains accurate as written and is not reproduced verbatim again here to avoid duplicating unchanged material — see the Third Edition's Chapter 8.2 for the complete field-by-field reference; every field name and type there was spot-checked against the current `schema.prisma` for this edition and no drift was found.

8.3 Key relationships, plain English

Unchanged: one Customer → many Invoices; one Invoice → many InvoiceItems/Payments/Returns; one Vendor → many PurchaseBills; one RateList → many RateListItems (no relation to Product/Brand/Category); a Product referenced by invoice/return/purchase-bill line items and the stock ledger, with the ledger surviving a product's deletion via a name snapshot.

9 Complete Module Documentation

9.1 Invoices

Purpose, numbering, and business rules (place-of-supply mandatory, no duplicate product lines, fully-paid invoices locked, server-independent `isInterState`, transportation charge, FY-bounded editable date, editable "Bill To" customer) are all unchanged in shape since the Third Edition.

NEW THIS EDITION

Drag-to-reorder line items (2026-08-20) — the invoice line-item table now supports reordering rows by drag, purely a display/entry-order convenience with no effect on GST calculation or the stored data shape (line items were always an ordered list; this just lets a user fix that order without deleting and re-adding a row). **Length-bounded validation** (2026-08-18) — every text field on the invoice form (customer/product free-text names, notes, place-of-supply) is now checked server-side against explicit min/max lengths, not just a format regex, closing a gap where an empty-but-technically-valid or absurdly long string could previously reach the database. **A client-generated idempotency key** now accompanies invoice creation (Chapter 13), and every mutation on an invoice (create, edit, delete, payment, return, cancel-adjacent flows) now correctly busts the invoice-stats cache and every other dependent cached view via the fixed `bustCachePrefix()` (Chapter 16.1) — previously, several of these flows busted only the invoice list's own cache key and left sibling cached views (like `/api/invoices/stats`) stale until a hard refresh.

9.2 Customers

Unchanged. A customer with any active invoice cannot be soft-deleted; inline creation from the invoice screen and inline editing from Invoice Edit both remain as documented in the Third Edition. Server-side length bounds (2026-08-18) now apply to name/address/city/GSTIN-adjacent free-text fields on the customer form, matching the client's own `FormField` limits.

9.3 Products, Brands, Categories

The single shared low-stock definition (`src/lib/stockStatus.ts`) and the soft-delete blocking rules are unchanged.

NEW THIS EDITION

Bulk import of products from Excel/CSV (2026-08-20) — Products gained an import path modeled directly on the Rate List import pattern (Chapter 9.18/27.7): a file is parsed (ExcelJS for `.xlsx`, plain comma-split for `.csv`) into rows the user still reviews and edits before anything is saved, rather than writing straight to the database. The product form's **"Price" field was relabeled "List Price"** in the same window, to remove ambiguity against the separate `purchasePrice` field sitting right next to it on the same form.

IN PROGRESS — UNCOMMITTED AS OF 2026-08-31

A **stock-adjustment preview** is being added to the Product detail page's Adjust Stock dialog: as the user types a delta, the dialog now shows the current stock, the signed delta, and the resulting new stock figure, each with a color tone matching the direction of the change (increase/decrease). This is a pure UI affordance on top of the existing, already-audited `POST /api/products/[id]/adjust-stock` endpoint — it does not change what gets written to the ledger, the required-reason rule, or the `"manual"` movement type. As this sits in the working tree, not yet committed, treat it as a preview of intent rather than a description of shipped behavior until a future edition confirms it landed.

9.4 Payments (Received)

Unchanged core behavior — a payment's amount is always recalculated from the true sum of all payments, never incrementally adjusted, and the invoice status is derived fresh every time.

NEW THIS EDITION

Payment creation now carries a client-generated idempotency key (Chapter 13, 16), scoped to the parent invoice, so a retried "Record Payment" submission can no longer create a second payment row. The payment-creation transaction also gained an explicit timeout — Prisma's default proved too tight for this database's real-world latency under load, which had been causing intermittent mid-transaction failures (2026-08-26).

9.5 Returns / Credit Notes

Unchanged business rules (must be against a paid invoice, capped by paid amount and remaining returnable quantity, restores stock). Return creation now also carries an idempotency key. A new index on `Return.date` (Chapter 8.2) speeds up date-range filtering on the Credit Notes list and its `/stats` companion.

9.6 Vendors

Unchanged. The 2026-08-27 fix (Chapter 20) means Vendor Edit now correctly shows a not-found state instead of crashing when the vendor id in the URL no longer resolves (e.g. a stale bookmark to a permanently-deleted vendor).

9.7 Purchase Bills

Numbering, transport charge, cancel/un-cancel workflow, and FY-bounded `billDate` editing are all unchanged in shape.

NEW THIS EDITION

Email a purchase bill directly (2026-08-26) — `POST /api/send-purchase-bill` is a new route mirroring `/api/send-invoice`, reachable from the Purchase Bill detail page's Share action. It accepts a comma-separated list of up to 10 recipient addresses (so a bill can go to a vendor and their accountant in one send), each individually validated against a basic email-shape regex and length-capped, with embedded CR/LF rejected on the bill-number field to prevent SMTP header injection — the same defensive pattern `/api/send-invoice` / `/api/send-rate-list` already used. See Chapter 13.2 and 18.3.

Purchase Bill edit gained the same in-transaction optimistic-concurrency recheck

Invoice edit already had (2026-08-26) — previously, Purchase Bill edit checked `expectedUpdatedAt` before starting its write transaction but not again inside it, leaving a race window where two concurrent edits could both pass the earlier check and the second could silently clobber the first's item/stock changes. It now re-evaluates the same condition inside an `updateMany` whose WHERE clause is checked under the row's own lock, exactly matching the Invoice edit pattern documented in the Third Edition's Chapter 10.4. **Drag-to-reorder** (2026-08-20) also applies to the purchase-bill item table, same as invoices.

Purchase bills also gained a **duplicate-product-line guard** during the audit pass (`BIZ-001`, 2026-08-19), matching the equivalent check invoices already had — previously the same product could appear as two separate purchase-bill line items with no warning, silently splitting what should have been one combined quantity.

9.8 Payments (Made)

Mirrors the sales-side payment logic. Purchase payment creation now carries an idempotency key, same as sales payments (2026-08-26). A new index on `PurchasePayment.date` speeds up its list/stats routes.

9.9 Stock & the Stock Movement Ledger

Unchanged — see Chapter 11 for the full, still-accurate `StockMovementType` table.

9.10 Reports

NEW THIS EDITION

GST Summary and Purchase-by-category now aggregate in the database instead of loading every matching row into application memory and reducing it in JavaScript (2026-08-26) — the same DB-side aggregation approach the Third Edition already documented for the dashboard's Top Customers/Vendors widgets, now extended to these two report views. This is a pure performance change with no effect on the numbers reported.

9.11 GST Filing

Unchanged workflow (Sales Register, B2B/B2C split, Credit Notes, HSN Summary, ZIP or `.xlsx` download, gated by `requireGstFilingAccess()`).

IN PROGRESS — UNCOMMITTED AS OF 2026-08-31

Formula-injection hardening for the exported CSV/XLSX (Chapter 19.8) touches `gstFilingWorkbook.ts` and `gstFilingZip.ts` 's CSV writer directly — every text cell that could contain a customer/vendor/product name or note is now passed through `neutralizeFormulaCell()` before being written, so a cell beginning with `=`, `+`, `-`, or `@` can no longer be evaluated as a spreadsheet formula when the export is opened in Excel. This has not yet been committed.

9.12 Settings

Unchanged since the Third Edition's six-modal reorganisation. Length-bounded validation (2026-08-18) now applies server-side to the bank-detail and identity fields on this page, matching the client's own limits.

9.13 Recycle Bin

NEW THIS EDITION

The GST-retained document types (invoices, purchase bills, credit notes) are now capped per request with a "Load more" control instead of the Bin page always fetching the entire retained history in one unbounded request (2026-08-26) — because these three types are exempt from the 30-day auto-purge and can therefore accumulate indefinitely, an old, long-lived business's bin could grow into a genuinely large unbounded fetch every time the page loaded. Emptying the whole bin (`DELETE /api/bin/empty`) now shows the true total count across *all* retained items in its confirmation dialog, not just the count of whatever had been loaded onto the page so far. The bin purge routes themselves (`bin/route.ts` , `bin/empty/route.ts`) were also switched from a per-row sequential query loop to one batched query per entity type during the 2026-08-21 audit pass (`PERF-001`), and the admin activity log's own lazy 30-day purge now only runs on its first page load rather than on every page change.

Every other rule from the Third Edition — GST-safe indefinite retention for invoices/purchase bills/credit notes, the FK-safe purge order, `protectedReason` annotations — is unchanged.

9.14 Admin — Users & Activity Log

NEW THIS EDITION

The **admin activity log now uses the shared `Pagination` component** instead of a hand-rolled page-scroll implementation (2026-08-20) — it still keeps its own scroll-to-top-on-page-change behavior (documented as the app's general pagination convention in the Third Edition's Chapter 14.7), just now composed from the shared component rather than a parallel copy of the same idea. Admin users/permissions routes were also capped at 500 rows during the 2026-08-26 hardening pass, as a defensive bound rather than a functional change (no real deployment of this app is expected to approach that row count on either table).

9.15 Global Search

Unchanged — seven entity types, five results each, 250ms debounce with cancellation. Rate Lists remain outside Global Search's entity set.

9.16 Email

NEW THIS EDITION

A **third** send endpoint now exists alongside `/api/send-invoice` and `/api/send-rate-list` : POST `/api/send-purchase-bill` (2026-08-26). See 9.7 above and Chapter 18.3 for its full mechanics.

9.17 PDF Generation

NEW THIS EDITION

The invoice/purchase-bill PDF renderer was rewritten (2026-08-19, part of commit `3cea3f8`) as part of the audit-fix pass. See Chapter 17 for what changed in the rendering pipeline itself; the caching mechanism (`pdfCache.ts`) and the shared `generateInvoicePdfBlob()` entry point are unchanged in their external contract, so Rate List PDFs continue to reuse the identical pipeline without modification.

9.18 Rate Lists

Unchanged in every respect documented in the Third Edition (data model, item calculation, list/detail routes, PDF reuse, bulk import via paste/upload, Excel export, current gaps around

public share links and the paste-import form's missing `FormField` error UI). Rate List items also gained drag-to-reorder (2026-08-20), alongside invoices and purchase bills.

9.19 Configurable Document Numbering & the Indian Financial Year

Unchanged. No layout, prefix, or FY-boundary rule changed in this window. This remains the single mechanism behind invoice, purchase-bill, and credit-note numbering, exactly as the Third Edition describes it in full in its own Chapter 9.19 (carried forward without material change — see Chapter 12.9 below for the compliance reasoning, also unchanged).

9.20 Shared UI Building Blocks

NEW THIS EDITION

Four new shared components (2026-08-25): `SearchField`, `DateRangeFilter`, `HeaderActionsRow`, and `ToolbarField` — extracted from what had become near-duplicate hand-rolled toolbar markup on nearly every list and report page, and rolled out app-wide in the same commit. `ConfirmDialog` **now portals to** `document.body` (2026-08-20), and `Spinner` **overlays do too** (2026-08-25) — both fix the same underlying bug class: a dialog/overlay nested under an ancestor with a CSS transform or animation applied (e.g. an `animate-card` section) was getting trapped inside that ancestor's own stacking context, letting later page content paint over it. **Mobile form-control heights were synced** across

`Button` / `Input` / `Select` / `DatePicker` / `PasswordInput` / `PhoneInput` in the same window.

Dashboard bar-chart value labels were reworked to float above each bar's own top, scaled to 84% of the bar's max height with a background badge on hover, replacing a shared fixed label row that had been clipping against the chart's own horizontal-scroll container.

The UnitCombo/Modal conventions documented in the Third Edition's Chapter 9.20 are unchanged and still accurate.

10 Invoice Flow — Step by Step

The thirteen-step flow, the transportation-charge mechanics (10.1), the configurable number format (10.2), the FY-bounded editable date (10.3), and the editable "Bill To" customer (10.4) are all unchanged in substance since the Third Edition. Two additions apply on top of that unchanged flow:

- **Idempotency.** Step 8 ("Submit → `POST /api/invoices` ") now sends a client-generated key alongside the request body. If the exact same key is submitted again for the same customer/parent context — a network retry after a timeout, or a double-tap that slipped past the disabled-button guard — the server recognizes it as the same submission rather than creating a second invoice. A genuine key collision against a *different* invoice is rejected outright (409), so a real duplicate payment is never silently dropped in the name of idempotency.
- **Drag-to-reorder.** Between steps 3 and 6, line items can now be reordered by drag before submission — a pure entry-order convenience with no effect on the GST calculation described below in Chapter 12.

The "why a transaction matters" explanation, the Serializable-transaction-with-retry mechanism for number generation, and the server-independent `isInterState` derivation are all unchanged and remain accurate as written in the Third Edition.

11 Stock Flow & the Movement Ledger

Every rule in this chapter is unchanged since the Third Edition: the closed, specific `StockMovementType` union with no generic "adjustment" catch-all, the `batchAdjustStock()` batched-delta mechanism, the single shared low-stock definition in `src/lib/stockStatus.ts`, the soft-delete-vs-permanent-delete asymmetry for products with binned purchase-bill history, and the admin-only `DELETE /api/stock-movements?type=stock-ledger` bulk-clear.

Movement type	documentType	When it happens
<code>sale</code>	invoice	A new invoice is created
<code>sale_edit_reverse</code> / <code>sale_edit_apply</code>	invoice	An invoice is edited — old quantities reversed, then new quantities applied
<code>sale_delete_restore</code>	invoice	An invoice is soft-deleted
<code>sale_bin_restore</code>	invoice	A deleted invoice is restored from the Recycle Bin
<code>purchase</code>	purchase_bill	A new purchase bill is created
<code>purchase_edit_reverse</code> / <code>purchase_edit_apply</code>	purchase_bill	A purchase bill's items are edited
<code>purchase_cancel</code> / <code>purchase_uncancel</code>	purchase_bill	A purchase bill is cancelled / un-cancelled
<code>purchase_delete_restore</code>	purchase_bill	A purchase bill is soft-deleted (skipped if already cancelled)
<code>purchase_bin_restore</code>	purchase_bill	A deleted purchase bill is restored (skipped if cancelled)
<code>return</code>	credit_note	A credit note is issued against an invoice
<code>return_delete_reverse</code>	credit_note	A credit note is soft-deleted
<code>return_bin_restore</code>	credit_note	A deleted credit note is restored
<code>manual</code>	manual	A manual stock adjustment after a physical count (mandatory reason)

IN PROGRESS — UNCOMMITTED AS OF 2026-08-31

The `manual` movement type's own creation flow (`POST /api/products/[id]/adjust-stock`) is unchanged; what's in the working tree is purely a client-side preview added to the Adjust Stock dialog before submission (Chapter 9.3) — the ledger row it eventually writes is identical in shape to what the Third Edition already documented.

12 GST Logic

The core per-line formula (discount before GST), the CGST+SGST vs. IGST decision via `deriveIsInterState()`, discount treatment, the transportation-charge exclusion from the item-level tax split, the Reverse Charge disclosure-only flag, commercial rounding via `computeRoundOff()`, the worked example, credit-note GST-rate preservation, and the numbering-as-compliance-control reasoning (12.9) are all confirmed unchanged and still accurate as written in the Third Edition's Chapter 12 — no GST arithmetic, rate derivation, or rounding rule changed in this window. This is worth stating plainly rather than silently: a document titled "GST billing system" whose tax math was touched by a hardening pass deserves an explicit "no, it wasn't" as much as a chapter documenting what did change.

The one adjacent change: purchase bills gained the duplicate-product-line guard already used by invoices (Chapter 9.7), which affects what *rows* can exist on a purchase bill, not how tax is computed on any individual row.

13 API Documentation (Full Reference)

Every endpoint below was confirmed to exist in the current codebase. "Auth" shows the minimum permission guard applied. All of `/api/**` additionally sits behind `middleware.ts`'s default-deny baseline.

13.1 Sales

Method / Path	Auth	Purpose
GET/POST /api/invoices	Session / Write	List (paginated/searchable/sortable) / Create — auto-numbers, decrements stock, computes transport-charge GST server-side. POST now accepts a client-generated idempotency key.
GET /api/invoices/stats	Session	Summary totals over all matching invoices, independent of the list route's current page.
GET/PUT/DELETE /api/invoices/[id]	Session / Write	View / edit (reverses+reapplies stock) / soft-delete.
POST /api/invoices/[id]/payment	Write	Record a payment; idempotency-key-guarded.
PUT /api/invoices/[id]/payment/[paymentId]	Write	Edit an existing payment.
GET/POST /api/invoices/[id]/returns	Session / Write	List/create a credit note; POST idempotency-key-guarded.
DELETE /api/invoices/[id]/returns/[returnId]	Write	Soft-delete a credit note.
GET/POST /api/customers	Session / Write	List / create customer.
GET/PUT/DELETE /api/customers/[id]	Session / Write	View / edit / soft-delete.
GET /api/payments , GET /api/payments/stats	Session	All sales payments, paginated / stats.
GET /api/credit-notes , GET /api/credit- notes/stats	Session	All credit notes, paginated / stats.
GET/POST /api/rate-lists , GET/PUT/DELETE /api/rate-lists/[id]	Session / Write	Unchanged from the Third Edition.
POST /api/rate-lists/parse-import	Write	Unchanged.
POST /api/send-rate-list	Write, rate-limited (20/15min)	Unchanged.

13.2 Purchases

Method / Path	Auth	Purpose
GET/POST /api/vendors , GET/PUT/DELETE /api/vendors/[id]	Session / Write	Unchanged. Vendor edit now checks <code>r.ok</code> before treating a fetch response as vendor data (2026-08-27).
GET/POST /api/purchase-bills	Session / Write	List / create — POST now idempotency-key-guarded.
GET /api/purchase-bills/stats	Session	Unchanged.
GET/PUT/DELETE /api/purchase-bills/[id]	Session / Write	PUT now re-checks the optimistic-concurrency lock <i>inside</i> the write transaction, not just before it (2026-08-26); detail/edit pages now check <code>r.ok</code> on fetch (2026-08-27).
POST /api/purchase-bills/[id]/payment	Write	Idempotency-key-guarded.
GET /api/purchase-bills/payments , .../payments/stats	Session	Unchanged.
POST/DELETE /api/purchase-bills/upload	Write	Unchanged.
POST /api/send-purchase-bill	Write, rate-limited (20/15min)	New (2026-08-26). Emails a purchase-bill PDF to up to 10 comma-separated recipients. Same Gmail-credential resolution, CR/LF injection guard on the bill number, and 10MB PDF cap as the other two send routes — see Chapter 18.3.
GET /api/purchase-reports	Section: reports_purchases	Purchase-by-category now aggregates via one <code>groupBy</code> in the database instead of an in-memory reduce (2026-08-26).

13.3 Catalogue

Unchanged endpoint set from the Third Edition (products, brands, categories, adjust-stock). Product bulk-import (2026-08-20) is a client-side parse feeding the existing `POST /api/products` create calls, not a new dedicated import endpoint — mirroring how Rate List import works (Chapter 9.18).

13.4 Reports & Filing

Unchanged endpoint set. `GET /api/reports?type=gst-summary` now aggregates by month in the database instead of loading every invoice into memory (2026-08-26) — same output shape, different implementation.

13.5 System & Admin

Unchanged endpoint set, with the Bin routes' internal implementation switched to batched per-entity-type queries (2026-08-21) and the GST-retained-type listing within `GET /api/bin` now paginated with a load-more cursor rather than unbounded (2026-08-26) — the route's external shape (query params, response fields, `daysLeft` / `protectedReason` annotations) is otherwise unchanged.

UPDATE — PAGINATION PAIRING COUNT

The list-route/stats-route pairing (list route returns `{data, total}`), a companion `/stats` route reuses the identical `buildXWhere()`) remains the pattern for the same nine endpoints documented in the Third Edition — no new entity gained this pairing in this window. `/api/send-purchase-bill` is not a list route and has no such pairing, matching `/api/send-invoice` / `/api/send-rate-list` .

14 Frontend Architecture

14.1 Pages & layouts

Unchanged — every authenticated page sits under `DashboardShell`.

14.2 State management

Still no Redux/Zustand — `useFetch(url)` backed by the shared in-memory cache in `src/lib/useCache.ts` remains the entire data layer. What changed this edition is documented fully in Chapter 16.1: `bustCachePrefix()`'s matching logic was fixed at the root, closing a real, live bug rather than a hypothetical one.

14.3 Forms & validation

The shared `rules.* / validate() / validateForm()` pattern, the edit-form-vs-create-form disable-button convention, and the primitive component set (`Input` , `Select` , `Textarea` , `FormField` , `PasswordInput`) are unchanged in shape.

NEW THIS EDITION

Every validation rule that previously checked only format now also checks length (2026-08-18) — `src/lib/validation.ts`'s `rules.*` and every per-entity `validateXInput()` gained explicit min/max bounds for name, address, city, bank fields, and password, applied identically on the client `FormField`s and the server route in the same commit. This closes a class of bug distinct from a format violation: a field that is syntactically valid (a GSTIN-shaped string, say) but empty, whitespace-only, or implausibly long was previously able to reach the database because no rule actually bounded its length, only its shape.

14.4 The app-wide Modal component

Unchanged — `variant="fullscreen"` remains the only variant any call site actually uses, portaled to `document.body`, with the `footer / form="..."` convention exactly as the Third Edition describes.

14.5 Shared UI components

Component	Purpose / what changed
Button	Unchanged this edition.
Input / Select / Textarea / FormField	Mobile heights synced (2026-08-25); server-side length bounds now back every rule these components submit against.
UnitCombo	Unchanged.
ConfirmDialog	Now portals to <code>document.body</code> (2026-08-20) — fixes it rendering trapped inside a transformed/animated ancestor.
Modal	Unchanged.
Pagination	Unchanged this edition; the admin activity log switched onto it (2026-08-20, see Chapter 9.14).
Spinner / FloatingSpinner	Overlay variants now portal to <code>document.body</code> too (2026-08-25), same fix as <code>ConfirmDialog</code> .
SearchField	New (2026-08-25) — a shared debounced search input, replacing per-page copies.
DateRangeFilter	New (2026-08-25) — a shared month/year or from-to date filter control.
HeaderActionsRow	New (2026-08-25) — a shared right-aligned action-button row for list/report page headers.
ToolbarField	New (2026-08-25) — a shared labeled-filter-field wrapper used inside list toolbars.

14.6 Rate List forms

Unchanged — see the Third Edition's full write-up, still accurate.

14.7 Pagination and mobile breakpoints

The three-breakpoint system (1024px sidebar/topbar drawer threshold, 768px form-field compact-mode, 640px Button shrink tier) is unchanged in its own right. What's new: the topbar now condenses its controls into a `MoreMenu` below 480px (2026-08-18) — a fourth, narrower breakpoint layered on top of the existing three, specifically for the topbar's own icon-button row rather than the sidebar drawer or form-field sizing. Mobile form-control heights were separately synced across six components (2026-08-25, see 14.5).

14.8 Theming

Unchanged.

15 Backend Architecture

The "no controller/service/repository layering" architecture is unchanged. The helper-module table from the Third Edition remains accurate, with two additions:

Helper module	Responsibility
<code>apiAuth.ts</code>	The four permission-guard functions.
<code>invoiceCalc.ts</code> , <code>purchaseBillForm.ts</code> , <code>roundOff.ts</code> , <code>rateListForm.ts</code>	Shared GST/discount/rounding math.
<code>stockMovement.ts</code>	The single place every stock change must go through.
<code>invoiceReturns.ts</code>	Prevents editing an invoice below what's already returned.
<code>blobStorage.ts</code>	File-upload URL allowlisting and cleanup.
<code>activity.ts</code>	Writes to the audit log; never throws.
<code>rateLimit.ts</code>	In-memory abuse-prevention counters.
<code>crypto.ts</code>	Encrypts/decrypts sensitive settings fields.
<code>gstFiling.ts</code> / <code>gstFilingZip.ts</code> / <code>gstFilingWorkbook.ts</code>	Builds the GST filing report/ZIP/xlsx. Formula-safety wiring in progress, uncommitted.
<code>documentNumbering.ts</code>	Shared numbering/FY engine.
<code>stockStatus.ts</code>	The single shared low/out-of-stock definition.
<code>gstLocation.ts</code>	Server-side inter-state derivation.
<code>pdfCache.ts</code>	Client-side IndexedDB PDF blob cache.
<code>useIdempotencyKey.ts</code>	New (2026-08-26) — client-side hook minting a stable per-form-mount key for a mutating POST.
<code>formulaSafety.ts</code>	New, uncommitted — <code>neutralizeFormulaCell()</code> , a single function used by the export writers described in 9.11/19.8.

15.1 Per-entity query builders

Unchanged — the same set of `xQuery.ts` modules and the same "hand-rolling `where` / `orderBy` logic in a route handler that already has a query-builder module is not permitted" rule. File uploads remain magic-byte validated and allowlisted, unchanged.

16 Data Flow

The overall request/response/cache diagram (browser → `useFetch` → cache-or-fetch → API route → Prisma → Postgres → JSON response → cache update) is unchanged. Every mutation still calls `revalidateTag(tag, { expire: 0 })` and writes an Activity Log entry.

16.1 `bustCache` vs. `bustCachePrefix` — the bug that motivated this window's biggest commit

UPDATE — ROOT CAUSE FIXED, 2026-08-26

The Third Edition (Chapter 16.1) had already documented and fixed one instance of this bug class:

`bustCache(url)` only ever invalidates the one exact URL string it's given, and `bustCachePrefix(prefix)` exists as the correct tool for busting a page that doesn't own the cached URL outright. What this edition found, triggered by a real user report — a credit note restored from the Bin not appearing on the Credit Notes list without a hard refresh — is that `bustCachePrefix()` **itself had a gap**: it matched a key equal to `prefix` or starting with `` + "${prefix}?`" + ``, but *not* a key starting with `` + "${prefix}/`" + ``. Busting `/api/invoices` correctly cleared the cached list itself, but silently missed sibling sub-routes like `/api/invoices/stats`, `/api/invoices/[id]/payment`, or any other path nested one level under the same base — because those cached keys start with `/api/invoices/...`, not `/api/invoices?...`.

This was not a narrow, one-page bug. Every route that busts a base resource's cache from outside that resource's own page — an invoice payment busting the invoices list, a purchase-bill cancel busting purchase-bills' stats, a bin restore busting the credit-notes cache — was silently missing its sibling sub-routes the same way. The fix (`src/lib/useCache.ts`) widened the match to also include any key starting with `` + "${prefix}/`" + ``:

```
export function bustCachePrefix(prefix: string) {
  for (const key of cache.keys()) {
    if (key === prefix || key.startsWith(`${prefix}?`) || key.startsWith(`${prefix}/`)) {
      // ...
    }
  }
}
```

Alongside the root fix, the same commit (`76412b3`) audited every existing mutation flow and added missing cache-bust calls that had been relying on the broken sibling-matching behavior without anyone noticing — invoice/purchase-bill payment, return, delete, edit, and cancel flows, plus brand/category rename (since products embed the brand/category's own name at read time, renaming one now correctly busts the products cache too, not just the brand/category list itself).

The rest of Chapter 16 — the `/stats` -route-has-its-own-cache-key rule, the cache tag list, the "no server-side data cache actually in effect today" statement — is unchanged and still accurate.

16.2 Idempotency keys — a second, related hardening (new this edition)

Cache staleness and duplicate submission are different bugs with the same root feeling ("I clicked once and something looks wrong"), so this edition's biggest commit fixed both in the same pass.

`src/lib/useIdempotencyKey.ts` mints a stable UUID once per form/dialog mount:

```
export function useIdempotencyKey() {
  const ref = useRef(crypto.randomUUID());
  function renew() {
    ref.current = crypto.randomUUID();
  }
  return { key: () => ref.current, renew };
}
```

The key is sent alongside a mutating POST for Invoice, PurchaseBill, Payment, PurchasePayment, and Return creation. The server-side lookup is scoped to the parent resource (an invoice's own payments, say), so a genuine key collision against a *different* parent is rejected with a 409 rather than silently treated as a duplicate and dropped — the intent is to stop an accidental double-submission, not to accidentally swallow a legitimate second payment that happens to reuse a stale key. `renew()` must be called after a successful submit on any form instance that stays mounted and could legitimately submit again (e.g. a payment dialog reused for a second, different payment) — otherwise the next real submission would be misidentified as a duplicate of the first.

17 PDF Generation

The overall pipeline — the on-screen HTML is "photographed" by `html2canvas` into an image, then pasted into an A4 jsPDF document by `generateInvoicePdfBlob()` (`src/lib/generateInvoicePdf.ts`) — is unchanged in concept.

UPDATE — RENDERER REWRITE, 2026-08-19

As part of the audit-fix commit (`3cea3f8` , closing `PDF-002` among other findings), the invoice/purchase-bill PDF renderer was reworked. `PDF-002` specifically covered `generateInvoicePdfBlob()` silently swallowing every failure with no logging — a render that failed partway through previously gave no signal to the user or the developer beyond a missing/blank download, making the failure mode essentially undiagnosable in production. The rewrite addresses this alongside general renderer improvements; the shared entry point's external contract (one function, targeting a `#invoice-print-area` -shaped container, returning a Blob) is unchanged, so every downstream consumer — the invoice/credit-note/purchase-bill detail pages, the Rate List print pipeline (17.3), and `pdfCache.ts` — required no changes of their own.

The multi-copy Original/Duplicate mechanism (17.1), the long-document page-splitting rule (17.2), Rate Lists' reuse of the identical pipeline (17.3), and the client-side `pdfCache.ts` IndexedDB cache keyed on entity + variant (17.4) are all unchanged in behavior and remain accurate as documented in the Third Edition.

18 Email System

The six-step flow (click send → generate PDF client-side → server checks role + rate limit → resolve Gmail credentials → 503 if unconfigured → Nodemailer sends via Gmail SMTP with an escaped HTML body) is unchanged. Credential resolution order (Business Settings first, environment-variable fallback second) is unchanged.

18.1 /api/send-invoice

Unchanged — 20 sends/15min per user, `requireWriteAccess()` -gated.

18.2 /api/send-rate-list

Unchanged.

18.3 /api/send-purchase-bill — new this edition

Added 2026-08-26 alongside the cache-invalidation hardening commit, reachable from the Purchase Bill detail page's Share action. Reading the route handler directly:

- `requireWriteAccess()` gate, then the identical per-user rate limit (20 sends/15 min, key `send-purchase-bill:${session.user.id}`) the other two senders use.
- Accepts a comma-separated `to` field — up to **10 recipients** in one send, each individually validated against a basic email-shape regex and capped at 254 characters. This is the one structural difference from `/api/send-invoice` / `/api/send-rate-list`, which each take a single recipient address; the purchase-bill sender was deliberately built to let a bill go to both a vendor and their accountant in one action.
- Embedded `\r / \n` or a bill number over 100 characters is rejected with a 400 before the value ever reaches the email `subject` header — the same SMTP header-injection guard `API-004`'s fix (Chapter 19) added to the other two senders.
- The attached PDF is capped at 10MB, mirroring the other two routes.
- Gmail credentials resolve in the same order:
`BusinessSettings.gmailUser` / `gmailAppPassword` first, then `GMAIL_USER` / `GMAIL_APP_PASSWORD`, with a 503 "Email not configured" response if neither is set.

19 Security

19.1 Authentication security

The bcrypt/dummy-hash/dual-rate-limit/session-cookie mechanisms are unchanged. This window's audit pass touched one authentication-adjacent surface:

FIXED — AUTH-001, 2026-08-19

`/api/auth/find-email` previously disclosed the full names of every user (staff and admin alike) to an unauthenticated caller, and was enumerable by substring — an attacker could iteratively narrow down who worked at the business and what their full name was, purely from the public find-email flow meant only to help a legitimate user locate their own masked email. This was closed during the audit pass; the route's now-narrower behavior (exact match only, rather than substring enumeration) matches what CLAUDE.md's own "Full-app audit" summary describes.

FIXED — AUTH-002, 2026-08-19

`admin/users/[id]`'s PUT handler was using a raw `getSession()` call for its self-edit path instead of routing through the shared `apiAuth.ts` guards — an informational-leaning inconsistency (the self-edit path was not itself exploitable) rather than a live vulnerability, but a real deviation from the "always go through the shared guard functions" rule documented since the Third Edition's Chapter 7.6. Fixed to match every other route's pattern.

19.2 Authorization

Unchanged — the four guard functions remain the sole enforcement mechanism, and the earlier (2026-07-03-era) privilege-escalation and unauthorized-write fixes documented in the Third Edition remain in place, unregressed.

19.3 Input validation / injection protection

The Prisma-parameterizes-everything, escaped-email-HTML, and magic-byte-file-upload protections are unchanged. What's new:

- **Length bounds** now back every validation rule, not just format (Chapter 14.3) — closing the class of gap where a syntactically-valid-but-empty or absurdly-long value could reach the database.
- **SMTP header injection was closed** on the email `subject` line across all three send routes (`API-004` , 2026-08-19) — an unsanitized, client-supplied string (an invoice number, a rate-list title, a purchase-bill number) previously flowed directly into the email subject with no

check for embedded `\r / \n`, which is the standard technique for injecting extra SMTP headers (e.g. a forged `Bcc`). All three senders now reject the request outright if the relevant field contains a newline or exceeds a defined length cap.

19.4 API trust-boundary fixes (new this edition)

FIXED — API-001, 2026-08-19

`PUT /api/invoices/[id]` previously let any write-access caller set an arbitrary, unvalidated `status` string on an invoice, decoupled from its actual `paidAmount` — meaning a caller could mark an invoice "paid" while it still had a genuine outstanding balance, silently hiding real dues from the Outstanding/Overdue reports. Status is now always recomputed server-side from `paidAmount / total`, matching the rule already documented for purchase bills in the Third Edition.

FIXED — API-002, 2026-08-19

`PUT /api/products/[id]` previously allowed `stock` to be changed directly through the general product-edit route, bypassing the audited `batchAdjustStock()` /ledger path the app was explicitly built to force every stock change through (documented as a hard rule since the Third Edition's Chapter 11 and CLAUDE.md's "Do Not" list). This has been closed — `stock` is rejected server-side on this route, not merely hidden on the client form.

FIXED — API-003, 2026-08-19

`PUT /api/purchase-bills/[id]` had the same status-trust gap as `API-001`, narrower in practice since "cancelled" is the one legitimate direct value a caller can already set — but any other arbitrary status string was previously accepted. Closed alongside `API-001`, in the same commit.

19.5 Blob-storage & asset validation

FIXED — SEC-004, 2026-08-19

`isPurchaseBillBlobUrl()` / `isLogoBlobUrl()` previously only checked a URL's general shape — any hostname matching the shared `*.public.blob.vercel-storage.com` suffix, which is common to *every* Vercel Blob store on the platform, not just this app's own — rather than an exact match against this app's specific store hostname. This is the exact-hostname-match hardening that `src/lib/blobStorage.ts` still documents today, already summarized in CLAUDE.md as the fix for a cross-store URL-spoofing risk (a URL pointing at someone else's unrelated Blob store, shaped to pass the old, looser check).

FIXED — SEC-005, 2026-08-19

Business-logo replacement previously relied on a client-fired, unawaited `DELETE` request to clean up the old blob — a fire-and-forget call the browser could fail to send (tab closed, navigation interrupted) with no server-side guarantee the old file was ever actually removed. Logo-blob cleanup is now performed server-side, inside the same `PUT /api/settings` request that writes the new logo URL, so cleanup and the write that makes it necessary are atomic from the server's point of view.

FIXED — SEC-003, 2026-08-21

`BusinessSettings.logoUrl` previously accepted any string via `PUT /api/settings` with no validation at all, despite being fetched directly by the browser and rendered as an `<img>` app-wide (every PDF header, the login page, the sidebar). Closed during the 14-phase audit pass.

19.6 Secrets at rest

Unchanged — AES-256-GCM via `src/lib/crypto.ts`, optional dedicated `ENCRYPTION_KEY`, `encv2: / enc: /legacy-plaintext` all still readable.

19.7 Rate limiting — still an honest limitation

Unchanged: in-memory fixed-window counters, no Redis/Upstash in production use.

`/api/send-purchase-bill` (new this edition) uses the identical rate-limit shape as the other two senders, so the app's total surface of rate-limited routes grew by one endpoint without changing the underlying mechanism's limitations.

19.8 CSV/XLSX formula-injection hardening — in progress, not yet committed

IN PROGRESS — UNCOMMITTED AS OF 2026-08-31

A new module, `src/lib/formulaSafety.ts` (currently untracked in git), exports one function:

```
const FORMULA_TRIGGER = /^[=+\-@]/;

export function neutralizeFormulaCell(value) {
  if (typeof value !== "string") return value;
  return FORMULA_TRIGGER.test(value) ? ` '${value}' ` : value;
}
```

A cell value beginning with `=`, `+`, `-`, or `@` can be interpreted as a formula by Excel when the file is opened — this is the standard "CSV/formula injection" (a.k.a. "spreadsheet formula injection") class of vulnerability: a business name, a customer note, or any other free-text field that ends up in an exported report could contain a value shaped like a formula (deliberately, from a malicious counterparty, or accidentally) and have it silently execute when a staff member opens the exported file in Excel. This risk is more acute for CSV than `.xlsx`, since CSV carries no per-cell type metadata to mark a value as literal text — a plain string write is directly exploitable — but the fix is applied to both as defense-in-depth, since a genuine `.xlsx` could later be re-saved or re-exported as CSV. The mitigation is the standard OWASP-recommended one: prefixing a triggering value with a leading apostrophe forces the cell to render as literal text rather than being evaluated.

As of 2026-08-31 this is wired into `gstFilingWorkbook.ts`, `gstFilingZip.ts`'s CSV writer, and `xlsxExport.ts` (confirmed via `git status` — all three show as locally modified, with `formulaSafety.ts` itself untracked), but **none of it is committed**. Treat this section as a description of code sitting in the working tree, not a description of what is currently deployed to production.

19.9 First-time setup safety, default-deny middleware

Unchanged — see Chapters 6.5 and 7.7.

20 Error Handling

The frontend banner-vs-toast distinction and the backend status-code table (401/403/404/400/409/413/429/503/500) are unchanged.

Optimistic-concurrency conflicts

Unchanged mechanism, now extended: as documented in Chapter 9.7, Purchase Bill edit gained the same in-transaction recheck Invoice edit already had (2026-08-26), closing the last money-mutating edit route that had only checked its concurrency guard before, not during, its write transaction.

Unguarded fetch responses — a real, fixed bug (new this edition)

FIXED — 2026-08-27, COMMIT BAEF4F2

Purchase Bill detail (`purchases/bills/[id]/page.tsx`), Purchase Bill edit (`.../[id]/edit/page.tsx`), and Vendor edit (`purchases/vendors/[id]/edit/page.tsx`) were all treating a fetch's response body as valid entity data without first checking whether the HTTP response was actually successful (no `r.ok` check). A 404 response — the shape returned for a deleted or invalid id — still has a JSON body (an error object), and that error object was being fed into the page's state as if it were the real bill/vendor, which then crashed the page's own rendering logic instead of showing a clean not-found state; the crash was caught by the app's error boundary (Chapter 20's frontend section, and the Third Edition's documentation of `(dashboard)/error.tsx`), but that boundary is a generic last-resort fallback, not a proper "this record doesn't exist" screen.

This was found via live browser testing, not a code-review pass — a useful reminder that a route handler returning the "correct" status code doesn't by itself guarantee every consumer of that response is checking it. Per `DEVELOPMENT_LOG.md`'s own updated convention: **any page that fetches a single entity by id must check `r.ok` before treating the response body as that entity** — a non-2xx body is not valid entity data, and should render a not-found state instead of flowing into the page as if it were.

Database, activity logging

Unchanged.

21 Deployment

The development and production flows are unchanged in shape (push to `main` → Vercel builds → migrations apply → deploy). One change to the build script this window:

UPDATE — RETRYING MIGRATE-DEPLOY WRAPPER, 2026-08-21

The Third Edition documented `"build": "prisma migrate deploy && next build"`. It is now `"build": "node scripts/migrate-deploy-retry.js && next build"` — a new script that retries `prisma migrate deploy` up to 5 times, 5 seconds apart, before giving up. This was added during the full-app audit pass to absorb Neon's free/low-tier compute cold-start latency, which had been causing an intermittent, non-deterministic build failure whenever a build happened to hit a cold database connection right at the migration step. The `prebuild` script (deleting stale `.next/dev` before every build, documented in the Third Edition) is unchanged.

The `postinstall` script (`prisma generate && node scripts/install-git-hooks.js`) is unchanged and remains mandatory — do not remove it. The Windows local-development note about the locked Prisma client DLL is unchanged.

22 Third-Party Services

The service table (Vercel, Neon, Vercel Blob, Gmail SMTP via Nodemailer, Razorpay's public IFSC directory) is unchanged in substance. Rate Lists' storage footprint (client-generated PDFs, never written to Vercel Blob) is unchanged.

UPDATE THIS EDITION

Purchase-bill emails are now a third client of the Gmail SMTP integration, alongside invoices and rate lists — no new third-party service was introduced, this is the same Nodemailer-via-Gmail-App-Password mechanism the Third Edition already documented, with a third internal caller.

23 Environment Variables

Unchanged from the Third Edition — `DATABASE_URL` , `NEXTAUTH_SECRET` , `NEXTAUTH_URL` , `ENCRYPTION_KEY` (optional), `GMAIL_USER` / `GMAIL_APP_PASSWORD` (optional), `BLOB_READ_WRITE_TOKEN` (optional). No new environment variable was introduced by any change in this window, including the new `/api/send-purchase-bill` route, which reuses the existing Gmail credential resolution described in Chapter 18.

24 Project Dependencies

The overall dependency list is largely unchanged — no new runtime package was added for any feature in this window (drag-to-reorder, product bulk-import, and the toolbar-component extraction are all built on existing React/CSS, not a new drag-and-drop or forms library). What changed:

Change	Detail
next	Confirmed at ^16.3.1 in the current package.json (up from 16.2.9 as of the Third Edition) — part of the audit-driven dependency bump that cleared every critical/high npm audit finding.
nodemailer	Confirmed at ^9.0.5 (up from the version current at the Third Edition) — a CVE-driven bump.
@auth/prisma-adapter	Removed entirely (2026-08-21) — the app is CredentialsProvider -only and this dependency was pulling in critical-severity @auth/core vulnerabilities (DEP-001) for zero actual functionality.
overrides block	New this edition — package.json now carries an npm overrides entry pinning @auth/core to 0.41.3 and forcing both @auth/core 's and next-auth 's own nodemailer peer dependency to resolve to the same version as the project's own top-level nodemailer ("nodemailer": "\$nodemailer"), rather than the ^7 ^8 range those packages independently peer-depend on.

WHY THE OVERRIDES BLOCK EXISTS

Both @auth/core and next-auth peer-depend on nodemailer ^7|^8 . The CVE-driven bump to nodemailer@9.0.5 (closing DEP-002 's six high-severity advisories) directly conflicted with that peer-dependency range on a clean install — npm install would either refuse to resolve or silently install a duplicate, incompatible copy. This surfaced as a genuine Vercel build failure (f1878aa , 2026-08-19), not a local-only annoyance: a stale local node_modules directory had been masking the conflict during day-to-day development, so only a clean install — exactly what Vercel's own build does on every deploy — actually hit it. Rather than downgrading nodemailer back to a version with known CVEs, or waiting on upstream next-auth / @auth/core to widen their own peer range, the fix pins the conflict away via npm overrides , forcing every copy of nodemailer in the dependency tree — including the ones @auth/core / next-auth would otherwise pull in themselves — to resolve to the same 9.0.5 the project already uses directly.

npm audit results referenced in CLAUDE.md (15 vulnerabilities, 3 critical/6 high, down to 2 accepted-moderate after the dependency work) are consistent with what this edition's spot-check of package.json confirms: next and nodemailer both at their bumped versions, @auth/prisma-adapter absent, and the overrides block present and correctly shaped to keep

the whole tree on one `nodemailer` version. Every other dependency in the Third Edition's table (`@prisma/client` / `prisma` , `next-auth` , `bcryptjs` , `jspdf` / `html2canvas` , `exceljs` , `jszip` , `@vercel/blob` , the CSS-Modules/Tailwind-present-but-unused styling toolchain, and the Vitest/Playwright test tooling) is unchanged.

25 Business Workflow — A Day in the Life

The narrative from the Third Edition — morning dashboard check, a purchase bill arriving, a Rate List being built and emailed to a prospect, that prospect becoming an invoice, payment recorded, a credit note issued, month-end vendor payment, GST filing at tax time, the Recycle Bin as a safety net — is unchanged in every particular. Two small additions worth narrating explicitly this edition:

1 When the owner pays a vendor's bill at month-end, that bill can now be emailed directly to the vendor (or the vendor's accountant, or both, in one send) straight from its detail page, the same way an invoice already could be — no separate step of downloading and manually attaching a PDF to an outside email client.

2 If two staff members happen to open the same purchase bill for editing within moments of each other, the second save is now caught and rejected with a clear conflict message rather than silently overwriting the first — closing the one gap in this protection that Purchase Bills still had as of the Third Edition (Invoice edit already had it).

26 Non-Technical Explanation

All four framings from the Third Edition — the shop owner's smart notebook, the risk-minded owner's safety net, the school student's library analogy, and the "never written a line of code" explanation of what a button click actually does — are unchanged and remain accurate descriptions of how the system behaves today. One addition worth stating in the same plain terms: this edition's biggest single piece of work was making sure that when something changes in one part of the app — a credit note being restored from the bin, say — every other screen that shows related information actually notices and updates itself, instead of occasionally needing a manual refresh to catch up. And every "Save" button that creates something new (an invoice, a bill, a payment) now has a safeguard against accidentally creating two of the same thing if a click gets sent twice, whether from a slow internet connection or an impatient double-tap.

27 Developer Guide

Sections 27.1 through 27.7 (first-time setup, first admin account, making a schema change, adding a new page, adding a new API route, adding a new paginated list endpoint, adding a Rate-List-adjacent bulk-entry feature) are unchanged from the Third Edition and remain the correct procedures.

27.8 Rules that must never be broken — updated this edition

Every rule from the Third Edition's equivalent list remains true. Three new rules, drawn directly from what this window's fixes established as hard conventions (see `DEVELOPMENT_LOG.md`'s Conventions section):

- **Any mutation route that touches a base resource must call `bustCachePrefix()` on it even when the change is only reflected in a sibling sub-route's cached response**, not the base route's own — e.g. recording a payment against an invoice must still bust `/api/invoices`, because `/api/invoices/stats` is a sibling sub-path of it, not because the payment changed the invoice list's own rows. Fixed at the root in `bustCachePrefix()` itself (Chapter 16.1), but any new code relying on cache-busting behavior should understand why this now works, not just that it does.
- **A create endpoint a client can plausibly retry or double-submit should accept a client-generated idempotency key**, scoped to its parent resource — follow the Invoice/PurchaseBill/Payment/PurchasePayment/Return pattern (`useIdempotencyKey.ts`, Chapter 16.2) for any new money- or document-creating route.
- **Any page that fetches a single entity by id must check `r.ok` before treating the response body as that entity** — a non-2xx body is not valid entity data. Fixed on Purchase Bill detail/edit and Vendor edit (Chapter 20); apply this check to every future single-entity fetch as a matter of course.

Two further conventions, already implicit in the Third Edition's practice but now made explicit in the project's own working notes, are worth restating here since they governed several of this window's fixes: never reset a "saving" flag on a success path that ends in `router.push` (leave it true until the page actually unmounts), and every Settings section must continue to save independently — a save action should only ever touch, validate, or send the fields for the section actually being edited.

28 Future Improvements

This chapter, as in the Third Edition, reflects the project's actual, current Pending Tasks and Known Issues — taken directly from `CLAUDE.md` as of 2026-08-31 — not a fresh audit's speculative wishlist.

Carried over and still genuinely open

- Multi-user simultaneous-edit notifications ("someone else is editing this invoice right now") — still deferred pending a design decision on a real-time channel (polling or WebSockets), which the app doesn't currently have. The underlying optimistic-concurrency conflict detection this would make proactive is now complete on every money-mutating route, including Purchase Bill edit as of this window — the notification layer on top of it remains the open piece.
- Scheduled/automatic reminder emails for overdue invoices, building on the email infrastructure that now spans three send routes (invoice, rate list, and — new this edition — purchase bill).
- AI & automation opportunities (still open, unchanged): automatic GST-rate/HSN suggestions on new products, and dashboard anomaly flags drawing on the same historical ledger/report data already collected.

Still open from the Third Edition, re-confirmed this edition

- **Provision a real test database.** `.env.test.example` still exists; `tests/api/**` and `tests/e2e/**` are still written and pass type-checking/lint, but — confirmed again as of this edition — have never actually executed against a live database. This audit window's own substantial set of fixes (Era 12) shipped without new regression tests for the same underlying reason: there is still no test database to run them against.
- **Rate Lists follow-up:** a public, token-based share link (so a customer can view/download without logging in) is the one remaining deliberately-deferred gap — Bin restore/permanent-delete for rate lists shipped before this edition's window and is no longer on this list.
- **Wire up `FormField` /error-state UI for `RateListItemsTable`'s paste-import** — its submit button is still gated purely on `!pasteText.trim()` with no field-level error surfaced on an empty submit, unlike every other quick-add modal in the app (the Categories/Brands Add & Rename modals had this exact gap fixed well before this edition's window). Not touched by any commit in this window.

New as of this edition

- **Land the uncommitted work.** The CSV/XLSX formula-injection hardening (`formulaSafety.ts` , Chapter 19.8) and the Product detail page's stock-adjustment preview (Chapter 9.3) both sit in the working tree, verified against `git status` as of 2026-08-31, and should be committed and re-verified once landed rather than assumed shipped from this document alone.
- **Expand test coverage further** once a real test database exists — this remains blocked on the same open item above, not on further test-writing; the existing coverage inventory (unit: `validation.ts` , `invoiceCalc.ts` , `roundOff.ts` , `gstLocation.ts` , `stockStatus.ts` , `purchaseBillForm.ts` , `listQuery.ts` , `documentNumbering.ts` , `rateListForm.ts` ; integration: customers, vendors, invoices (create only), bin, purchase-bills, purchase-bill-payments, returns) has not been extended in this window and does not yet cover brands/categories/products, rate-lists, credit-notes' own list route, sales payments' invoice-payment sub-route, settings, admin/*, search, the three send-* email routes, gst-filing, export-xlsx, or the bin's permanent-delete/empty routes.

Resolved since the Third Edition — no longer applicable

- The `bustCachePrefix()` sibling-sub-path gap (Chapter 16.1) — fixed at the root, 2026-08-26, not merely documented as a known limitation.
- Purchase Bill edit's missing in-transaction optimistic-concurrency recheck — fixed 2026-08-26, closing the one asymmetry the Third Edition's Chapter 20 had implicitly left between Invoice and Purchase Bill edit.
- Unguarded fetch responses on Purchase Bill detail/edit and Vendor edit — fixed 2026-08-27.
- Every critical/high `npm audit` finding present as of the Third Edition — cleared during the 2026-08-19/21 dependency work (Chapter 24); 2 accepted-moderate findings remain, per `CLAUDE.md` .

This is the final chapter of the Science Hub Bible, Fourth Edition.

Verified against d:\nextApps\science-hub as of 2026-08-31. Prior editions: First (2026-06-24 era), Second (2026-07-24), Third (2026-08-17).